

```

1 grammar org.example.games.Games with org.eclipse.xtext.common.Terminals
2
3 generate games "http://www.example.org/games/Games"
4
5 Model:
6     elements+=TopLevel*
7     game = (Game)?
8 ;
9
10 TopLevel:
11     Game
12     | Setup
13     | Rule
14     | Player
15 //     | Decisions
16     | IntVar
17     | Script
18     | Condition
19     | Decision
20 ;
21
22
23
24 Game:
25     'game' name=ID '{'
26         'setup'      '{' setup=[Setup|ID]          '}'
27         'rule'       '{' rule=[Rule|ID]            '}'
28 //         'decisions' '{' decisions=[Decisions|ID] '}'
29         'script'     '{' script=[Script|ID]         '}'
30     '}'
31 ;
32
33 Setup:
34     'setup' name=ID '{'
35         vars+=IntVar*
36         ('timelimit' limitName=ID)? // <-- plain ID, no [IntVar|ID]
37         'player1' '{' player1=[Player|ID] '}'
38         'player2' '{' player2=[Player|ID] '}'
39     '}'
40 ;
41
42
43
44 enum VarKind: VAR | RESOURCE | MILITARY;
45
46 IntVar:
47     kind=VarKind
48     type=Type
49     (
50         'var'      name=ID
51         | 'Resource' '[' name=ID ']'
52         | 'Military' '[' name=ID ']'
53     )
54     '=' init=Expression ';'
55 ;
56
57 Type:
58     name=('String' | 'int' | 'boolean')

```

```

59 ;
60
61 Assignment:
62     target=LValue '=' expression=Expression ';'
63 ;
64
65
66
67 Player:
68     'player' name=ID '{'
69         vars+=IntVar*
70         'health' '=' health=INT
71         'resources' '{' resources+=IntVar* '}'
72         ('military' '{' militaryResources+=IntVar* '}')?
73     '}'
74 ;
75
76
77 Rule:
78     'rule' name=ID '{'
79         'win' ':' winCondition=[Condition|ID]
80         'lose' ':' loseCondition=[Condition|ID]
81     '}'
82 ;
83
84 Condition:
85     'condition' name=ID boolexp=Expression ';'
86 ;
87
88
89
90 //Decisions:
91 //     'decisions' name=ID '{'
92 //         decisions+=[Decision|ID] (',' decisions+=[Decision|ID])*
93 //     '}'
94 //;
95
96 Decision:
97
98     'decision' name=ID '(' me=[Player|ID] ',' other = [Player|ID] ')' body=Block
99 ;
100
101
102
103 Block:
104     '{' statements+=Statement* '}'
105 ;
106
107 ExpressionStmt:
108     expr=Expression ';'
109 ;
110
111 Statement:
112     Conditional
113     | PrintStmt
114     | ExpressionStmt
115     | IntVar
116     | Assignment

```

```

117 ;
118
119 PrintStmt:
120     'print' value=STRING ';'
121 ;
122
123
124
125 Script:
126     'script' name=ID '{'
127         turns+=Turns*
128     '}'
129 ;
130
131 Turns:
132     'turn' index=INT ':'
133         player=[Player|ID]
134         decision=[Decision|ID]
135     ';'
136 ;
137
138
139
140 Conditional:
141     'if' '(' condition=Expression ')'
142         thenBlock=Block
143         ('elif' '(' elifConditions+=Expression ')' elifBlocks+=Block)*
144         ('else' elseBlock=Block)?
145 ;
146
147
148
149
150 LValue:
151     {PlayerVarLValue} player=[Player|ID] '.' varName=ID
152     | {VarLValue} varName=ID
153 ;
154
155
156
157 Expression:
158     OrExpression
159 ;
160
161 OrExpression returns Expression:
162     AndExpression ({OrExpression.left=current} 'or' right=AndExpression)*
163 ;
164
165 AndExpression returns Expression:
166     ComparisonExpression ({AndExpression.left=current} 'and' right=ComparisonExpression)*
167 ;
168
169 ComparisonExpression returns Expression:
170     Addition ({Comparison.left=current} op=('==' | '!=' | '<' | '>') right=Addition)*
171 ;
172
173 Addition returns Expression:
174     Multiplication ({Plus.left=current} '+' | {Minus.left=current} '-') right=Multiplication)*

```

```
175 ;
176
177 Multiplication returns Expression:
178   Primary (({Mult.left=current} '*' | {Div.left=current} '/') right=Primary)*
179 ;
180
181 Primary returns Expression:
182   '(' Expression ')'
183   | {IntConstant} value=INT
184   | {BoolConstant} value=('true'|'false')
185   | {LValueExpr} target=LValue
186   | {Not} '!' expression=Primary
187 ;
```

```

2  * generated by Xtext 2.32.0
4  package org.example.games.generator
5
6  import org.example.games.games.*
7
8  import org.eclipse.emf.ecore.resource.Resource
9  import org.eclipse.xtext.generator.AbstractGenerator
10 import org.eclipse.xtext.generator.IFileSystemAccess2
11 import org.eclipse.xtext.generator.IGeneratorContext
12
13 /**
14  * Generates code from your model files on save.
15  *
16  * See https://www.eclipse.org/Xtext/documentation/303\_runtime\_concepts.html#code-generation
17  * «game.name.toFirstUpper»
18  */
19 class GamesGenerator extends AbstractGenerator {
20
21     override void doGenerate(Resource resource, IFileSystemAccess2 fsa, IGeneratorContext
22 //     fsa.generateFile('greetings.txt', 'People to greet: ' +
23 //         resource.allContents
24 //             .filter(Greeting)
25 //             .map[name]
26 //             .join(', '))
27     val model = resource.contents.head as Model
28
29     val playerArr = newArrayList
30
31     for (game : model.elements.filter(Game)) {
32         fsa.generateFile(
33             game.name.toFirstUpper + ".java",
34             game.compile()
35         )
36     }
37 }
38
39 def CharSequence compile(Game game) {
40     ...
41     package my.generated;
42
43     import java.util.Map;
44     import java.util.HashMap;
45
46     public class «game.name.toFirstUpper»Game {
47
48
49         «FOR v : (game.eContainer as Model).elements.filter(IntVar)»
50         «v.extendIntVar»
51         «ENDFOR»
52
53
54         «FOR p : (game.eContainer as Model).elements.filter(Player)»
55         «p.extendPlayer»
56         «ENDFOR»
57
58
59         «game.setup.extendSetup»
60

```

```

61
62     «FOR c : (game.eContainer as Model).elements.filter(Condition)»
63     «c.extendCondition»
64     «ENDFOR»
65     «game.rule.extendRule»
66
67
68     «FOR d : (game.eContainer as Model).elements.filter(Decision)»
69     «d.extendDecision»
70     «ENDFOR»
71
72
73     «game.script.extendScript»
74
75     public static void main(String[] args) {
76         «game.name.toFirstUpper»Game g = new «game.name.toFirstUpper»Game();
77         g.runScript();
78     }
79 }
80
81 }
82
83 def extendIntVar(IntVar I) {
84     «I.type.name» «I.name» = «I.init.extendExpression»;
85 }
86
87 }
88
89 def CharSequence extendSetup(Setup st) {
90     public int timelimit = «IF st.limitName != null»«st.limitName»«ELSE»0«ENDIF»;
91
92     public Player «st.player1.name» player1 = «st.player1.name»;
93     public Player «st.player2.name» player2 = «st.player2.name»;
94 }
95
96 }
97
98
99 def extendScript(Script script) {
100     «FOR t : script.turns»
101     «t.extendTurns»
102     «ENDFOR»
103 }
104
105 }
106
107 def extendTurns(Turns turns) {
108     System.out.println("Turn: «turns.index»");
109     System.out.println("Player Name: «turns.player.name»");
110     «turns.decision.name». «turns.decision.name»_func
111 }
112
113 }
114
115 def CharSequence extendRule(Rule rule) {
116     boolean winCond() { return «rule.winCondition.name»(); }
117     boolean loseCond() { return «rule.loseCondition.name»(); }
118 }

```

```

119     '''
120 }
121
122 def CharSequence extendCondition(Condition cond) {
123     '''
124     boolean «cond.name»() {
125         return «cond.boolexp.extendExpression»;
126     }
127     '''
128 }
129
130 def CharSequence extendDecision(Decision decision) {
131     '''
132     static class «decision.name» {
133         public static void «decision.name»_func(
134             Player «decision.me.name» me,
135             Player «decision.other.name» other) {
136
137             «decision.body.extendBlock»
138         }
139     }
140     '''
141 }
142
143 def extendBlock(Block block) {
144     '''
145     «FOR s : block.statements»
146         «s.extendStatement»
147     «ENDFOR»
148     '''
149 }
150
151 def CharSequence extendPlayer(Player pl) {
152     '''
153     class Player«pl.name» {
154         «FOR i : pl.vars»
155             «i.extendIntVar»
156         «ENDFOR»
157
158         public int health = «pl.health»;
159         private Map<String, Integer> resources = new HashMap<>();
160         private Map<String, Integer> military = new HashMap<>();
161
162         public Player«pl.name»() {
163             «FOR r : pl.resources»
164                 resources.put("«r.name»", «r.init.extendExpression»);
165             «ENDFOR»
166             «FOR m : pl.militaryResources»
167                 military.put("«m.name»", «m.init.extendExpression»);
168             «ENDFOR»
169         }
170     }
171
172     public Player«pl.name» «pl.name» = new Player«pl.name»();
173     '''
174 }
175
176 def dispatch extendStatement(Conditional cond) {

```

```

177     '''
178     if («cond.condition.extendExpression»){
179         «cond.thenBlock.extendBlock»
180     }
181     «FOR i : 0 ..< cond.elifBlocks.size»
182         else if («cond.elifConditions.get(i).extendExpression»)
183         {
184             «cond.elifBlocks.get(i).extendBlock»
185         }
186     «ENDFOR»
187     «IF cond.elseBlock != null»
188         else {
189             «cond.elseBlock.extendBlock»
190         }
191     «ENDIF»
192     '''
193 }
194
195 def dispatch extendStatement(ExpressionStmt exp) {
196     '''
197     «exp.expr.extendExpression»
198     '''
199 }
200
201 def dispatch extendStatement(IntVar i) {
202     '''
203     «i.extendIntVar»
204     '''
205 }
206
207
208 def dispatch extendStatement(PrintStmt p) {
209     '''
210     System.out.println("«p.value»");
211     '''
212 }
213
214 def dispatch extendStatement(Assignment A) {
215     '''
216     «A.target.extendLVal» «A.expression.extendExpression»;
217     '''
218 }
219
220 def dispatch extendLVal(PlayerVarLValue plVal) {
221     '''«plVal.player.name» «plVal.varName» = '''
222 }
223
224 def dispatch extendLVal(VarLValue rVal) {
225     '''«rVal.varName» = '''
226 }
227
228 def dispatch extendExpression(Expression exp) {
229     '''
230     /* Unknown Expression Type */
231     '''
232 }
233
234 def dispatch extendExpression(OrExpression orExp) {

```



```
235     '''(«orExp.left.extendExpression» || «orExp.right.extendExpression»)'''
236   }
237
238   def dispatch extendExpression(AndExpression andExp) {
239     '''(«andExp.left.extendExpression» && «andExp.right.extendExpression»)'''
240   }
241
242   def dispatch extendExpression(Comparison cmpExp) {
243     '''(«cmpExp.left.extendExpression» «cmpExp.op» «cmpExp.right.extendExpression»)'''
244   }
245
246   def dispatch extendExpression(Plus pl) {
247     '''(«pl.left.extendExpression» + «pl.right.extendExpression»)'''
248   }
249
250   def dispatch extendExpression(Minus mn) {
251     '''(«mn.left.extendExpression» - «mn.right.extendExpression»)'''
252   }
253
254   def dispatch extendExpression(Mult ml) {
255     '''(«ml.left.extendExpression» * «ml.right.extendExpression»)'''
256   }
257
258   def dispatch extendExpression(Div div) {
259     '''(«div.left.extendExpression» / «div.right.extendExpression»)'''
260   }
261
262   def dispatch extendExpression(IntConstant IC) {
263     '''(«IC.value»)'''
264   }
265
266   def dispatch extendExpression(BoolConstant B1) {
267     '''«B1.value'''
268   }
269
270   def dispatch extendExpression(Not nt) {
271     '''!«nt.expression.extendExpression'''
272   }
273
274   def dispatch extendExpression(LValueExpr LVE) {
275     '''«LVE.target.extendLVal'''
276   }
277
278 }
279
```